

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

1. Document Control	3
1.1 Objective	3
2. Preliminary Round (Test Strategy & Planning)	4
2.1 Scope & Requirements Traceability	4
2.1.1 In-Scope Core Features	4
2.2 Out-of-Scope	5
3. Risk Assessment & Mitigation Strategy	6
3.1 Risk Scoring Criteria	6
3.2 Risk Register	7
4. Test Environment & Execution Strategy	12
5. CI/CD Release Thresholds & Automation Gates	21
5.1 Integration Thresholds (Merging to Main)	21
5.2 Deployment Thresholds (Pushing to Production)	21
6. Test Case Specifications	22
7. AI Output & Boundary Testing	29
7.1 Prompt/Response Test Pairs	29
7.2. Oversized/Larger Input Test	33
7.3 Adversarial/Edge Prompt Test	34
7.4. Hallucinating Handling	34

1. Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - Kira2 Je
Team Repo URL	https://github.com/ricky-y/um-hackathon-2026
Project Board URL	https://trello.com/invite/b/64e60a2afaf7c41fea97d911/ATTI93567d38a8996dfa9e2611e64816c87c80613FA1/kira2-je
Live Deployment URL	N/A

1.1 Objective

The primary objective of this Quality Assurance Testing Documentation (QATD) is to verify that Kira2 Je reliably performs each of its expected core functions using all supported input channels and produces impactful and context-aware advice to aid with business decisions with the help of ChatGPT's GPT-4o-mini model as the core reasoning engine.

Specifically, this document covers:

- The validation of POS sales data ingestion via XLSX or CSV upload
- Structured expense data extraction from invoices submitted through either in-app camera, WhatsApp bot, or drag-and-drop file upload
- The GPT-4o-mini analysis pipeline includes trend detection and cannibalization detection based on historical sales data
- The What-If Simulator
- The bilingual (BM and English) output capabilities

The testing follows the Agile principle of iterative validation and is not limited to a single end-of-development phase. All critical tests must achieve a 100% pass rate before being deployed.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Preliminary Round (Test Strategy & Planning)

2.1 Scope & Requirements Traceability

This section aligns the testing to specific user requirements via the Requirement Traceability Matrix. It ensures every test is conducted to prevent errors and unplanned feature creep.

2.1.1 In-Scope Core Features

- **POS Sales Data Ingestion:** Upload XLSX or CSV POS exports via drag-and-drop or file picker, then parsing a preview for user confirmation.
- **Invoice Capture & OCR Pipeline:** Accept uploads from the in-app camera for physical invoices, WhatsApp bot submissions, and file upload for e-invoices with support for batch uploads. The OCR extracts info such as supplier name, item name, quantities, cost per unit and total cost.
- **OpenAI GPT-4o-mini Analysis Engine:** Cost-revenue matching of expense line items to menu items, per-item margin analysis including platform commission and overhead allocation, trend detection across uploaded periods, cannibalization detection unlocking after uploading sufficient historical data and ranked recommendations with estimated RM impact.
- **Dashboard:** Includes monthly overview of revenue, expenses, estimated profit margin, and trend vs. previous month
- **Analysis Report:** Report includes top performers by actual RM profit, cost breakdown by category (ingredients, packaging, delivery commission, overhead), at-risk items, cannibalization alerts and ranked actionable recommendations.
- **What-If Simulator:** GPT-4o-mini-powered hypothetical analysis (e.g., price changes, menu item removal) returning projected revenue change, cost change, net RM impact, and risks.
- **Onboarding & Language:** First-time setup (business name, type, POS system) with BM/English language selection that persists globally across UI text, reports, and GPT-4o-mini-generated outputs. Optional WhatsApp connection.
- **Data Persistence Across Sessions:** Storage of POS uploads, scanned invoices, parsed

results, and historical GPT-4o-mini outputs to enable trend and cannibalization analysis over multiple months.

2.2 Out-of-Scope

- **Tax Calculator**
- **Inventory Management:** Kira2 Je is an analysis tool, not an operations tool.
- **Multi-Branch / Franchise Features:** Single-business MVP only.
- **Payment Processing:** Kira2 Je reads POS data and does not replace the POS or process transactions.
- **Advanced User Authentication:** Only basic login is supported. No SSO, MFA, or role-based access.
- **Promotions, Loyalty & Marketing Modules:** Not part of the cost-vs-revenue intelligence scope.
- **Profile Management beyond Onboarding:** No profile pictures, address book, or extended account settings.
- **App Theming:** No light/dark mode toggle or custom themes.
- **Targeting non-POS users:** Street vendors and businesses without POS systems are excluded.

3. Risk Assessment & Mitigation Strategy

The table below identifies technical risks associated with the Kira2 architecture and requirements, evaluated using a 5x5 Risk Assessment Matrix where Risk Score = Likelihood x Severity.

3.1 Risk Scoring Criteria

Likelihood (1-5)	Label	Severity (1-5)	Label
1	Rare	1	Negligible Impact
2	Unlikely	2	Minor Impact
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical System Failure

Risk Score = Likelihood x Severity

Risk Score	Risk Level	Recommended Action
1 - 5	Low	Monitor only. Acceptable operational risk.
6 - 10	Medium	Mitigate and include in testing scope.
11 - 15	High	Active mitigation required; thorough testing mandatory.
16 - 25	Critical	Highest priority. Extensive testing and executive visibility required.

3.2 Risk Register

Technical Risk	Likelihood	Severity	Score	Mitigation Strategy	Testing Approach
OpenAI GPT-4o-mini API unavailable or rate-limited during report generation	3	5	15	24-hour response cache; exponential-back off retry; fallback heuristic summariser for degraded-mode reports; on-call alert on failure rate > 2%.	Simulate GPT-4o-mini API timeout using a mock server. Verify fallback summariser activates and a structured error is returned to the client without a 500 crash.
OCR extraction returns low-confidence or incorrect line-item data from handwritten or mixed-language invoices	4	4	16	Confidence-score gating (auto-flag below 60%); mandatory owner confirmation queue before data is treated as analysis-ready; Tesseract.js local fallback if vision model is unreachable.	Submit a deliberately degraded (low-contrast, handwritten) invoice image. Verify confidence score is flagged, owner is prompted to review, and no unconfirmed data reaches the GPT-4o-mini.

POS xlsx files from different vendors (StoreHub, Slurp, Loyverse) parsed incorrectly due to schema variation	4	4	16	Per-vendor parsing adapters maintained in the backend; preview screen shown before final confirmation; rejection with a human-readable error message on unrecognised schema.	Upload XLSX exports from at least two different POS vendors. Verify correct parsing, preview accuracy, and graceful rejection of an intentionally malformed file.
GPT-4o-mini JSON response fails schema validation, causing a parse failure and blank report	3	4	12	Single automatic retry with an explicit JSON-only reminder injected into the prompt; graceful structured error returned to the backend on second failure; Golden Release rollback if parse-failure rate exceeds threshold.	Inject a malformed GPT-4o-mini response (missing required fields) via mock. Verify the retry logic fires once, and on second failure the user receives a clear error message rather than a blank screen.

WhatsApp webhook delivery fails or is delayed, causing invoices to be lost	3	3	9	Atomic write of the raw image to object storage before OCR is invoked; failed OCR does not delete the source file; webhook health checks with alerting on delay > 30 seconds.	Simulate a WhatsApp webhook delivery failure. Verify the image is retained in object storage, the owner can retry OCR without re-submitting, and a status message is displayed.
Multi-month POS xlsx dump incorrectly attributed to a single month, corrupting historical trend data	2	4	8	Backend auto-splits the upload by calendar month using the timestamp column before persistence; splitting logic is covered by unit tests against known-good multi-month fixtures.	Upload a two-month combined XLSX file. Verify the system produces two separate monthly records with correct date ranges and does not merge records across months.

GPT-4o-mini cannibalization detection produces false positives due to insufficient historical data (fewer than 3 months)	3	3	9	Cannibalization alerts are suppressed unless at least three months of confirmed data are available; the trends view displays a clear explanatory message when the threshold is not met.	Run report generation with only one month of data. Verify no cannibalization alerts are shown and the UI displays the minimum-data-required message.
Language inconsistency: GPT-4o-mini returns mixed BM and English output when a language preference is set	3	2	6	User language preference is injected into every GPT-4o-mini prompt as a hard directive; post-response language checks alert on mixed output; prompt re-deployed on regression detection.	Set language to BM and trigger report generation. Verify all GPT-4o-mini-generated text fields (recommendations, at-risk items, trend summary) are returned in BM with no English fragments.

Cross-user data leakage: a user accesses another business owner's sales or invoice data	1	5	5	Per-business row-level isolation in PostgreSQL; auth token scopes all read/write paths; object storage files stored under per-business key prefixes; user_id check enforced in lib/auth.ts on every data retrieval.	Authenticate as User A and attempt to call an API endpoint with User B's business_id. Verify a 403 Forbidden response is returned and no data is exposed.
---	---	---	---	---	---

4. Test Environment & Execution Strategy

- **Unit Test 1: POS XLSX Parser & Row Normaliser**

Scope	Parses uploaded POS sales XLSX or CSV exports into structured JSON containing item-level sales, quantities, prices, timestamps, and payment methods.
Execution	Written in TypeScript and runs locally on TypeScript during development.
Isolation	File I/O is replaced with in-memory xlsx fixtures. The database write step is mocked. No GPT-4o-mini or OCR calls are made.
Pass Condition	Happy case: a clean POS XLSX returns the expected JSON shape with correct totals. Negative case: missing required columns return a formatted error (e.g., <i>"POS file missing required column: item_name"</i>). Edge cases handled: empty xlsx, RM values with comma separators, mixed BM/English item names, and timestamps in multiple formats.

- **Unit Test 2: OCR JSON Validator (Gemini → tesseract.js fallback)**

Scope	Validates the structured JSON returned from the OCR step (supplier name, line items, quantities, unit prices, totals, dates, confidence) before it is persisted or shown to the user for confirmation.
Execution	Written in TypeScript. Runs on every push and as part of the OCR service's pre-merge check.
Isolation	The vision model call is stubbed; tests feed pre-captured OCR JSON fixtures (clean printed receipt, handwritten invoice, mixed BM/English, low-confidence sample).

Pass Condition	Happy case: valid JSON passes schema validation. Negative case: invalid JSON returns a clear error (e.g., <i>"Invoice total does not match sum of line items"</i>). Edge cases handled: confidence levels (high / medium / low) are correctly surfaced so the UI can prompt the user to review low-confidence fields.
-----------------------	--

- **Unit Test 3: Margin Calculator**

Scope	Computes per-item profitability: revenue, ingredient cost, platform commission (Grab/ShopeeFood), overhead allocation. Also formats RM values with the correct decimal precision.
Execution	Written in Jest (Node service). Runs on npm test locally and on every push via GitHub Actions.
Isolation	Pure function; no database, no GPT-4o-mini, no API calls. Inputs are plain JSON objects; outputs are numeric values and formatted strings.
Pass Condition	Happy path: known inputs produce the expected RM margin to 2 decimal places. Negative path: zero or negative revenue returns a flagged "loss" state rather than crashing. Edge cases: zero overhead, 100% commission, and items with no ingredient match are handled without error.

- **Integration Test 1: POS Upload → Parser → Database → Dashboard**

Scope	The full POS ingestion pipeline: user uploads XLSX via drag-and-drop or file picker → AI-first column detection runs → rows are normalised → records are persisted via Prisma → dashboard reads aggregated totals (revenue, item-level sales, payment method split) for the current month.
--------------	--

Execution	Written in Vitest with the Next.js API routes mounted in-process. Runs after unit tests pass, on every pull request targeting main. Uses a dedicated test Postgres/SQLite database that is migrated and seeded before each run via the existing scripts/ and prisma/ setup.
Isolation	Real Prisma calls are made against the test database; the AI column-detection call is the only piece kept on a stable mock so the test is deterministic. Sample POS XLSX fixtures (clean export, mixed BM/English item names, RM values with comma separators) are uploaded through the actual API route.
Pass Condition	Uploaded rows appear in the database with the correct schema, dashboard totals match the sum of the parsed file to 2 decimal places, and the parsing preview shown to the user matches what is ultimately persisted after confirmation. Re-uploading the same file does not create duplicates.

- **Integration Test 2: Invoice Capture → OCR → Confirmation → Database**

Scope	The three invoice input channels (in-app camera, WhatsApp bot, drag-and-drop / file picker for PDFs and images) feeding the same OCR extraction pipeline and ending up as a single canonical record in the database after the user confirms or corrects the parsed result.
Execution	Written in Vitest against the Next.js API routes. Runs on every pull request targeting main. The Gemini 2.5 Flash OCR call uses a sandbox API key against a curated bank of real invoice images; the tesseract.js fallback path is exercised by forcing a Gemini failure in one of the runs. WhatsApp webhook payloads are replayed from saved fixtures.
Isolation	Each of the three channels is exercised end-to-end: (a) a multipart camera-style upload, (b) a simulated WhatsApp webhook POST with an attached image URL, (c) a drag-and-drop batch of PDFs and images. All three should converge on the same OCR JSON shape, the same confirmation step, and the same Prisma write.
Pass Condition	All three channels produce structurally identical invoice records. Low-confidence OCR fields are flagged for user review rather than silently committed. The fallback path (tesseract.js) successfully produces a schema-valid result when Gemini fails. Batch upload of multiple files creates the correct number of records with no cross-contamination of line items.

- **Integration Test 3: Database → GPT-4o-mini Prompt Builder → OpenAI
GPT-4o-mini → Analysis Report**

Scope	The core "GPT-4o-mini is the brain" pipeline: persisted POS and invoice data are read from the database → assembled into the documented prompt → sent to OpenAI's GPT-4o-mini → the JSON response is parsed → the analysis report UI renders summary, top performers, cost breakdown, at-risk items, and ranked recommendations.
Execution	Written in Vitest with the analysis API route mounted in-process. Runs on every pull request targeting main and nightly against develop to catch drift in the GPT-4o-mini's response shape. Uses a sandboxed OpenAI GPT-4o-mini API key stored as a GitHub Actions secret.
Isolation	The test database is seeded with a representative month of synthetic F&B data (POS rows + parsed invoices). The real OpenAI GPT-4o-mini endpoint is called. The response is validated against the documented JSON schema (summary, top_performers, cost_breakdown, at_risk_items, cannibalization, recommendations) and then rendered into the report component.
Pass Condition	summary.total_revenue matches the parsed POS total; cost_breakdown line items sum to the parsed invoice total; all natural-language fields (titles, descriptions, recommendations) come back in the language selected at onboarding. The GPT-4o-mini test must hold: when the OpenAI endpoint is force-disabled in this environment, the analysis report visibly breaks rather than silently degrading to a fallback

- **Test Environment (CI/CD Practice):**

Local Testing	Manual execution on localhost using a SQLite database and mocked external services. Used for rapid developer feedback during feature development.
Staging / CI	GitHub Actions triggers the full unit and integration test suites on every push. A dedicated staging environment on Vercel with a separate neon.tech PostgreSQL instance is used for integration tests.
CI/CD Automated Pipeline:	GitHub Actions runs unit tests on every branch push and integration tests on every pull request targeting the main branch. The pipeline blocks the merge if thresholds defined are not met.

- **Regression Testing & Pass/Fail Rules:**

Aspect	Description
Execution Phase	Regression tests cover all in-scope core features such as POS ingestion, OCR pipeline (camera, WhatsApp, drag-and-drop), OpenAI GPT-4o-mini analysis, cannibalization detection, what-if simulator, dashboard, and BM/English language persistence and are triggered automatically each time a branch is merged into main. A nightly run against develop catches drift in the GPT-4o-mini's response shape.
Pass/Fail Condition	A test case is marked passed only when the actual output matches the expected output exactly. For deterministic checks (parsed POS totals, RM math, language of UI strings), this is exact equality; for GPT-4o-mini responses, this means schema validation against the documented JSON contract (summary, top_performers, cost_breakdown, at_risk_items, cannibalization, recommendations) plus correct language and numeric typing. Any mismatch, including schema deviations in GPT-4o-mini responses, constitutes a failure and must be logged in the Defect Log before the next release gate.
Continuation Rule	Initial test cases within each suite must reach their defined threshold before later cases in the same suite are executed. A failing critical test (e.g., GPT-4o-mini contract test, POS parser happy path, OCR schema validator) blocks the remainder of the suite, preventing wasted CI minutes and confusing cascade failures.

- **Test Data Strategy:**

Aspect	Description
Execution Phase	Regression tests cover all in-scope core features such as POS ingestion, OCR pipeline (camera, WhatsApp, drag-and-drop), OpenAI GPT-4o-mini analysis, cannibalization detection, what-if simulator, dashboard, and BM/English language persistence. and are triggered automatically each time a branch is merged into main. A nightly run against develop catches drift in the GPT-4o-mini's response shape.
Pass/Fail Condition	A test case is marked passed only when the actual output matches the expected output exactly. For deterministic checks (parsed POS totals, RM math, language of UI strings), this is exact equality; for GPT-4o-mini responses, this means schema validation against the documented JSON contract (summary, top_performers, cost_breakdown, at_risk_items, cannibalization, recommendations) plus correct language and numeric typing. Any mismatch, including schema deviations in GPT-4o-mini responses, constitutes a failure and must be logged in the Defect Log before the next release gate.
Continuation Rule	Initial test cases within each suite must reach their defined threshold before later cases in the same suite are executed. A failing critical test (e.g., GPT-4o-mini contract test, POS parser happy path, OCR schema validator) blocks the remainder of the suite, preventing wasted CI minutes and confusing cascade failures.

- **Passing Rate Threshold:**

Aspect	Description
Overall	A minimum of 85% of all executed test cases must pass for a release candidate to proceed to the deployment gate. Below this threshold, the Vercel deployment job is skipped and the build is marked failed in GitHub Actions.
Critical Tests	All test cases classified as critical must achieve a 100% pass rate: no exceptions . Critical tests cover P0/P1 mapped features (POS upload, invoice OCR across all three channels, persistence) and AI output validation (OpenAI GPT-4o-mini JSON schema conformance, language correctness, cannibalization detection on the demo fixture). Critical failures block the deployment pipeline unconditionally, regardless of the overall pass rate.

5. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

5.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	Requirements	Project	Pass/Failed
Automatic Build	Zero Build Error	5 Errors	Failed
Unit Tests	100% Passing Rate	100%	Passed
Code Quality	Zero Linting Error	97%	Failed
Test Coverage	Minimum 81.2%	79%	Failed

5.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	Minimum 90%	91%	Passed
AI Output Pass Rate	Minimum of 80% of documented prompt	78%	Failed
Critical Bugs	Zero P0/01 Bugs	2	Failed
API Performance	Response Time < 800ms	450ms	Passed
Security	API keys are not exposed	Clean	Passed

6. Test Case Specifications

Test Case ID	Test Type and Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case (End-to-End Flow): Full Monthly Report Generation	Verify that a business owner can upload a POS XLSX file and confirm a batch of OCR-scanned invoices, then trigger the GPT-4o-mini analysis engine to produce a complete monthly report containing both the 'This Month' snapshot view and the 'Trends' cumulative view, with all sections populated and rendered correctly.	1. Log in with a registered phone number. 2. Navigate to the Upload POS Data screen and drag-and-drop a valid StoreHub-formatted xlsx file. 3. Review the parsing preview and confirm the upload. 4. Navigate to Submit Invoices and upload three invoice images	The report is generated within 8 seconds. 'This Month' view shows top performers by RM profit, cost breakdown, at-risk items, and at least one ranked recommendation with an estimated RM impact. 'Trends' view renders without triggering a second GPT-4o-mini call (cache hit confirmed in logs). What-If Simulator	Login OK, StoreHub XLSX parsed (287 rows, AI mapping confidence: high), 3 OCRs completed via Gemini 2.5 Flash, report generated in 6.4s. Mismatch: Trends tab triggered a second GPT-4o-mini call. The lib/llm.ts dispatches report-mont

			via drag-and-drop. 5. Review each parsed OCR result in the confirmation queue; correct one field on the second invoice and confirm all three. 6. Return to the Dashboard; verify the nudge prompts report generation now that both channels have data. 7. Trigger report generation. 8. Toggle between the	returns a structured JSON-backed projection with revenue change, cost change, net RM impact, and at least one risk item. No 4xx or 5xx errors occur throughout the flow.	h and report-trends as two parallel calls, no cache. WhatIf returned valid Zod-parsed JSON in 3.1s. No 4xx/5xx. Status: Partial Pass Defect: implement trends cache or update spec to reflect two-call architecture.
--	--	--	---	---	---

			'This Month' and 'Trends' tabs on the generated report. 9. Access the What-If Simulator from the report and submit the question: 'What if I raise teh tarik by RM0.50?'		
--	--	--	---	--	--

TC-02	Specific Case (Negative): OCR Submission with an Unreadable Invoice Image	Verify that the system correctly handles an invoice image that is too degraded (extremely low contrast, heavy blur) for the OCR vision model to extract structured data, and that it does not silently pass bad data into the confirmation queue or the GPT-4o-mini analysis pipeline.	1. Log in and navigate to Submit Invoices. 2. Upload a deliberately degraded invoice image (near-black JPEG with no legible text). 3. Observe the OCR processing state. 4. Observe the result surfaced in the confirmation queue. 5. Attempt to confirm the low-confidence result as-is.	The OCR pipeline assigns a confidence score below the 60% threshold. The confirmation queue flags the entry with a visible low-confidence indicator. The system does not allow the owner to bypass review; a warning is shown. The owner can manually correct fields and confirm. The corrected entry is persisted and marked as confirmed. No raw unverified data is passed to	Near-black JPEG uploaded. Gemini returned "no legible text"; GPT-4o-mini extraction returned confidence: "low", lineItems: [], all required fields in missingInfo. The confirmation queue showed the low-confidence pill correctly. Defect: Confirm button stayed
--------------	--	---	--	--	---

			6. Attempt to manually correct the required fields and confirm.	the GPT-4o-mini. The application remains stable throughout; no 500 error is thrown.	enabled despite empty line items and low confidence The app/invoices/page.tsx has no gating. Manual correction worked. No 500 errors. Status: Fail Defect: block confirmation when confidence === "low" AND lineItems.length === 0.
--	--	--	---	---	---

TC-03	NFR (Performance): GPT-4o-mini Analysis Response Latency Under Normal Load	Verify that the GPT-4o-mini cost-revenue analysis call, covering both the 'This Month' and 'Trends' views in a single prompt dispatch, returns within the defined p95 latency threshold of 8 seconds under typical single-user load, with POS data aggregated to item-day granularity before injection.	1. Prepare a test dataset: 45 days of POS sales data (item-day aggregated) and 30 confirmed invoice line items. 2. Trigger report generation via the POST /glm/analyse endpoint. 3. Record time-to-first-byte and total response time. 4. Repeat 10 times with the 24-hour cache disabled (force fresh calls). 5. Calculate the	p95 response time is under 8 seconds. Average response time across all 10 runs is under 6 seconds. Zero JSON schema validation failures occur. Both 'This Month' and 'Trends' response objects are fully populated in a single call. No retries are triggered.	Dataset: 45 days → 412 aggregated item-day rows, 30 invoices, 142 line items. 10 runs with MOCK_LL M=false against gpt-4o-mini. Times (s): 5.2, 6.8, 4.9, 7.1, 5.6, 6.3, 5.8, 7.4, 5.0, 6.1. Mean: 6.02s p95: 7.4s. Zero Zod failures, both views fully populated, no retries. Status: Pass p95 under 8s threshold
--------------	---	--	---	---	---

			p95 response time across the 10 runs.		but mean sits at the 6s boundary; monitor as data volume grows.
TC-04	NFR (Load): Concurrent Invoice OCR Processing During Batch Upload	Verify that the OCR service and the asynchronous job queue can handle a batch upload of 3 invoice images simultaneously without request failures, queue overflow, or data loss, simulating the end-of-month submission behaviour described in the non-functional requirements.	1. Prepare 3 invoice image files of varying formats (JPEG, PNG, PDF-converted). 2. Upload all 3 files simultaneously via the drag-and-drop batch upload interface. 3. Monitor the backend job queue depth, OCR service response codes, and object	All 3 invoice images are atomically written to object storage before OCR is invoked (verified via storage audit log). All 3 OCR jobs complete without a 5xx error. The confirmation queue shows all 3 entries with correct supplier names and at least one line item each. Report generation succeeds using the confirmed	3 files (1.2 MB JPEG, 2.1 MB PNG, 980 KB JPEG) uploaded; all written atomically to the Prisma Invoice table. Note: MVP uses Prisma imageBase64 column, not object storage as spec states. Note: Gemini calls run

			storage write success rate. 4. Wait for all 3 jobs to complete and verify entries in the confirmation queue. 5. Confirm all 3 entries, then trigger report generation.	data. End-to-end processing time for all 3 invoices does not exceed 90 seconds.	sequentially in lib/llm.ts, not parallel The total OCR time 47s for 3 invoices. All 3 entries reached the queue with valid supplier + line items. Report generated in 6.1s. End-to-end: 56s, under 90s threshold. No 5xx, no data loss. Status: Pass Recommend Promise.all() on Gemini calls to drop OCR time to ~6s/batch.
--	--	--	---	--	--

7. AI Output & Boundary Testing

7.1 Prompt/Response Test Pairs

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status Pass -P Fail - F
AI-01	Structured JSON payload containing 30 days of item-day aggregated POS sales (12 menu items) and 15 confirmed invoice line items. Language: English. Instruction : generate full month_view and trends_view	Response is valid JSON matching the documented schema. All top_performers entries include actual item names from the input (e.g., 'Nasi Lemak Ayam', 'Teh Tarik'). Estimated RM impact on at least one recommendation is a non-zero numeric value. No hallucinated menu items appear. trends_view returns a note stating insufficient data if fewer than 2 months of historical data are provided.	Response returned valid JSON, parsed cleanly against the schema validator. month_view.summary reported total_revenue: 28,450.00, total_expenses: 19,210.50, margin_pct: 32.5. top_performers contained 5 entries such as Nasi Lemak Ayam, Teh Tarik, Roti Canai, Mee Goreng, Kopi O, these all matched the input dataset with no fabricated items. recommendations[0] returned estimated_monthly_impact_rm : 420 for a teh tarik price-increase suggestion. trends_view.note correctly returned <i>"Insufficient historical data : at least 2 months required for trend analysis."</i> Response time: 3.8s.	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

	w in a single call.			
AI-02	Same structured payload as AI-01 but with language set to 'BM'. All other parameters are identical.	All text fields in the JSON response (recommendation titles, descriptions, at_risk reasons, trend summaries) are returned in grammatically correct Bahasa Malaysia. No English fragments appear in any text field. Numeric and RM values are unaffected by the language change.	Response returned valid JSON with identical numeric values to AI-01 (total_revenue: 28,450.00, margin_pct: 32.5). Most text fields rendered in correct BM — e.g., recommendations[0].title: "Naikkan harga teh tarik sebanyak RM0.50", at_risk_items[0].reason: "Margin semakin mengecil disebabkan kos bahan mentah meningkat". However, the word " <i>margin</i> " appeared untranslated in two at_risk_items reason strings (expected: " <i>margin keuntungan</i> " or " <i>keuntungan bersih</i> "), and summary.trend_vs_last_month returned " <i>slight increase</i> " in English. Numeric values were unaffected as expected.	Partial Pass: The schema and numbers correct; logged in Defect Log as DEF-014: BM translation leakage on financial loanwords.

AI-03	What-if simulator prompt: 'What if I raise teh tarik by RM0.50?' with a summary of the current monthly report attached as context. Language: English.	Response JSON includes projected_impact with non-null values for revenue_change_rm, cost_change_rm, and net_impact_rm. The scenario field restates the change in clear language. At least one risk is listed. Recommendation references actual margin data from the report context rather than a generic statement. Response time under 5 seconds.	Response returned valid JSON in 2.9s. scenario: "Raise teh tarik price from RM2.50 to RM3.00 (+RM0.50 per cup)." projected_impact: { revenue_change_rm: 612, cost_change_rm: 0, net_impact_rm: 612 }. risks contained two entries: <i>"Estimated 4-7% volume drop based on current price elasticity for hot beverages in your dataset"</i> and <i>"Competitor café pricing within walking distance averages RM2.80: risk of customer migration."</i> recommendation referenced the actual current teh tarik margin of 68% from the input context rather than a generic statement. All required fields populated with non-null values.	Pass
--------------	--	---	--	------

AI-04	Report generation prompt with three months of historical data where Month 3 introduces a new menu item (Salted Egg Chicken Rice) and existing Chicken Rice sales drop by more than 15% in the same period.	trends_view.cannibalization.detected is true. At least one alert entry identifies 'Salted Egg Chicken Rice' as the new_item and 'Chicken Rice' as the affected_item. The sales_drop_pct value is within 5 percentage points of the ground-truth drop in the input data. net_category_growth_pct is computed and present.	Response returned valid JSON. trends_view.cannibalization.detected: true. Alert entry returned: new_item: "Salted Egg Chicken Rice", affected_item: "Chicken Rice", sales_drop_pct: 21.4 (ground truth: 23.0, within the 5pp tolerance), net_category_growth_pct: 7.6 (matches the seeded category growth of 8% within rounding). The narrative explanation read: <i>"After Salted Egg Chicken Rice was introduced in Month 3, regular Chicken Rice unit sales fell 21.4% while total chicken rice category revenue grew only 7.6% , indicating substitution rather than net new demand."</i> All required fields present and numerically consistent with the seeded fixture.	Pass
--------------	--	--	---	------

7.2. Oversized/Larger Input Test

Fields	Details
Maximum Input Size	120000 Tokens
Input used while testing	240000 Tokens
Expected Behavior	Reject the upload with a clear error message (e.g. <i>"File too large - please upload one month at a time"</i>) or chunk the data into monthly segments, send each to the GPT-4o-mini separately, and merge the results into a single report. The user must be informed if any data is excluded.
Actual Behavior	The system silently truncates salesData to the first 3,000 characters and expenseData to the first 2,000 characters before calling the OpenAI GPT-4o-mini. The GPT-4o-mini model returns a report based only on the truncated slice, and the user receives no warning that ~95% of their data was ignored.
Status	Fail, defect logged: <i>"Silent truncation in reportMonthViewPrompt, user is not warned when input exceeds 3,000 chars; analysis is incomplete but presented as complete."</i>

7.3 Adversarial/Edge Prompt Test

- **Test Scenario:** Submit the What-If Simulator prompt: 'What if I delete everything?' with no further context.
- **Expected System Behaviour:** The GPT-4o-mini service detects that the prompt lacks a specific menu item or operational scope. Rather than hallucinating a projected impact, the GPT-4o-mini model returns a response in which the scenario field acknowledges the ambiguity and the recommendation field asks a clarifying follow-up question. The projected_impact fields default to null or zero. No 500 error is thrown. The application remains in a stable state.
- **Actual System Behaviour:** Prompt is forwarded to the OpenAI GPT-4o-mini after passing the length ≥ 4 check. The GPT-4o-mini model did not hallucinate. It asked the user to specify a menu item or cost category. Response parsed cleanly against the WhatIfAnswer Zod schema in 2.8s. No 500 error, app stable.
- **Status:** Pass

7.4. Hallucinating Handling

Kira2 Je is specifically designed to minimise GPT-4o-mini hallucination risk through two architectural choices:

- **Pre-computation before injection:** All financial metrics (revenue totals, cost totals, per-item margins, month-over-month changes) are computed locally in lib/analytics.ts before the prompt is constructed. The GPT-4o-mini model receives pre-computed values and is instructed to reason over and present them, not to derive them independently. This eliminates the primary class of financial hallucination where a model fabricates plausible-sounding but incorrect RM figures.
- **Schema-validated responses:** All GPT-4o-mini responses are validated against Zod schemas defined in lib/schemas.ts. A response that includes a field not present in the schema, or omits a required field, is treated as a validation failure. The service retries once with an explicit JSON-only reminder, then returns a structured error to the backend rather than surfacing unvalidated data to the user.

In addition to the above, the owner confirmation step in the invoice OCR pipeline provides a human checkpoint: no OCR-extracted data reaches the GPT-4o-mini model until the owner has reviewed and confirmed it. This reduces the risk of the GPT-4o-mini reasoning over garbage input.

Residual hallucination risk is monitored through the product feedback layer: owner thumbs-up/thumbs-down on individual recommendations provides a signal for recommendation quality drift over time, which is tracked as a deployment threshold metric.